

Overview

Welcome to Semantica

Core Concepts

Core Concepts

Modules

Choose the Right Module

Glossary

Guides

Architecture

Learning More

GraphRAG: Graph-Augmented Retrieval

Ingest Anything

LLM Integrations

Pipeline Builder

Multi-Agent Systems

Decision Intelligence

Context Graphs

Agent Memory

Provenance & Audit Trails

Semantic Extraction

Ontology Management

Reasoning & Rules

Export & Serialization

Deduplication & Entity Merging

MCP Server

SHACL Validation

Conflict Detection & Resolution

Change Management & Versioning

Policy Engine

Visualization

Distance Intelligence

Graph Analytics

Integrations

Agno Integration

CrewAI Integration

LangChain Integration

Docling Integration

Snowflake Integration

Databricks Integration

Salesforce Integration

Graph Stores

Apache AGE Graph Store

Vector Stores

pgvector Store

FAQ

FAQ

Community

Community

Community Projects

Contributing

Governance

Citation

License

Core Concepts

Core Concepts

The fundamental ideas behind Semantica: knowledge graphs, reasoning, provenance, and temporal intelligence explained.

🔔 New here? Start with [Getting Started](#) for hands-on examples, then return here for deeper understanding.

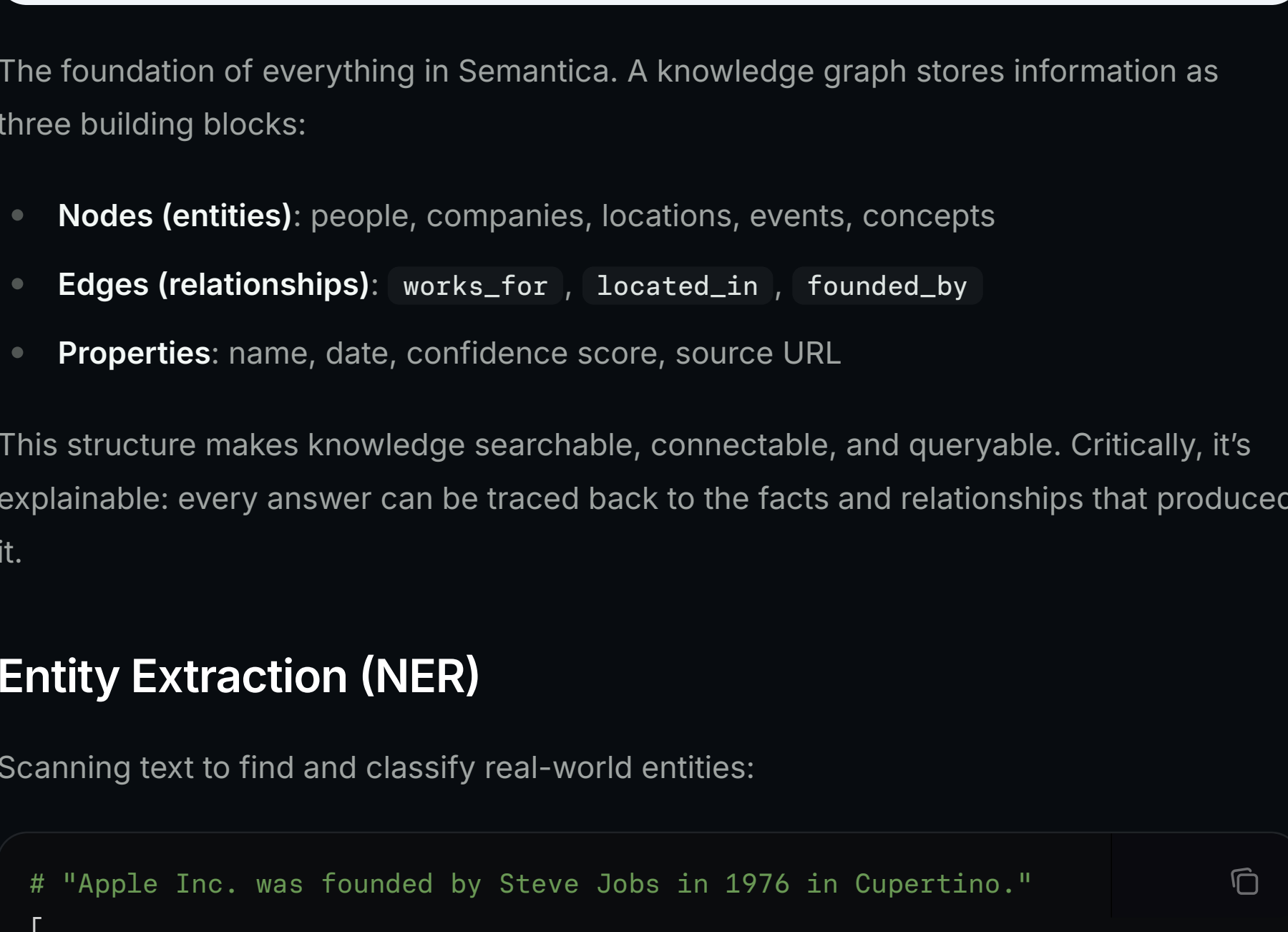
Semantica transforms unstructured data (documents, web pages, reports, databases) into **knowledge graphs**: structured representations that AI systems can query, reason about, and trace back to sources.

At its core, Semantica adds a context and semantic layer on top of your existing AI stack. It doesn't replace LangChain, LlamaIndex, or your LLM provider. It makes their outputs grounded, traceable, and auditable.

- **Context Layer.** Knowledge graphs, GraphRAG retrieval, semantic embeddings, and temporal intelligence ground every LLM response in structured, queryable facts.
- **Accountability Layer.** Provenance tracking, decision intelligence, conflict detection, and W3C PROV-O compliance make every claim in your AI stack auditable and explainable.
- **Extension Layer.** `PluginRegistry` and `MethodRegistry` let you replace or augment any component (ingestors, extractors, reasoning engines, backends) without changing framework code.

⚠️ This is **system-level explainability**, **not foundation-model explainability**. Semantica does not expose, reconstruct, or explain what happens inside the LLM foundation model. Its internal reasoning or chain-of-thought stays opaque, as it does for any external system. What Semantica explains is *outside* the model: the context and data fed in, the decision produced, its provenance, the relevant relationships, the policies applied, and the full execution trail. In short, Semantica explains and audits what the *AI system did*, not the foundation model's private internal reasoning.

Knowledge Graphs



The foundation of everything in Semantica. A knowledge graph stores information as three building blocks:

- **Nodes (entities):** people, companies, locations, events, concepts
- **Edges (relationships):** `works_for`, `located_in`, `founded_by`
- **Properties:** name, date, confidence score, source URL

This structure makes knowledge searchable, connectable, and queryable. Critically, it's explainable: every answer can be traced back to the facts and relationships that produced it.

Entity Extraction (NER)

Scanning text to find and classify real-world entities:

```
# "Apple Inc. was founded by Steve Jobs in 1976 in Cupertino."
[
  Entity(text="Apple Inc.", label="ORG",      start_char=0, end_char=10, confi
  Entity(text="Steve Jobs", label="PERSON",  start_char=26, end_char=36, confi
  Entity(text="1976",      label="DATE",     start_char=39, end_char=43, confi
  Entity(text="Cupertino", label="GPE",      start_char=47, end_char=56, confi
]
```

`NERExtractor(method=...).extract(text)` returns a list of `Entity` objects, each with a `label`, character offsets (`start_char` / `end_char`), a `confidence` score, and a `metadata` dict recording the extraction method. Three methods are available:

METHOD	SPEED	ACCURACY	REQUIREMENTS
"pattern"	⚡ Very fast	Moderate	No API key; regex-based
"nli"	Fast	High	Local ML model
"llm"	Medium	Highest	LLM provider; all 9 supported

Relationship Extraction

Finding how entities connect to each other:

```
jobs = Entity(text="Steve Jobs", label="PERSON", start_char=26, en
apple = Entity(text="Apple Inc.", label="ORG",    start_char=0,   end_char=10)

[
  Relation(subject=jobs, predicate="founded",    object=apple, confidence=
  Relation(subject=apple, predicate="located_in", object=Entity(text="Cuperi
]
```

`RelationExtractor(method=...).extract(text, entities=entities)` returns a list of `Relation` objects: typed subject-predicate-object triples (the endpoints are `Entity` objects) with confidence scores and source attribution. Extraction runs via pattern rules, ML models, or LLMs.

Knowledge Graph vs. Vector Store

Both store information for AI retrieval: but they're built for different jobs.

Knowledge Graph	Vector Store	GraphRAG (Both)
Stores structured facts as typed nodes and labeled edges. Answers questions that require understanding relationships between entities.		
STRENGTH		
Traversal		Multi-hop queries: "Who founded companies that Apple alumni later joined?"
Explainability		Every answer traces back to specific nodes and edges; no black-box retrieval
Temporal reasoning		Point-in-time queries, <code>valid_from</code> / <code>valid_until</code> windows, historical snapshots
Conflict detection		Two sources disagreeing on the same fact is surfaced and resolvable
Schema enforcement		SHACL validation catches constraint violations before they corrupt results
Use when: you need structured reasoning, provenance, compliance, or explainability.		

```
from semantica.kg import GraphBuilder, PathFinder

graph = GraphBuilder(merge_entities=True).build(
  {"entities": entities, "relationships": rels}
)
path = PathFinder().dijkstra_shortest_path(graph, "Steve Jobs", "Tim Cook")
```

Embeddings

Embeddings convert text into numerical vectors so AI systems can measure semantic similarity: finding related concepts even when the exact words differ.

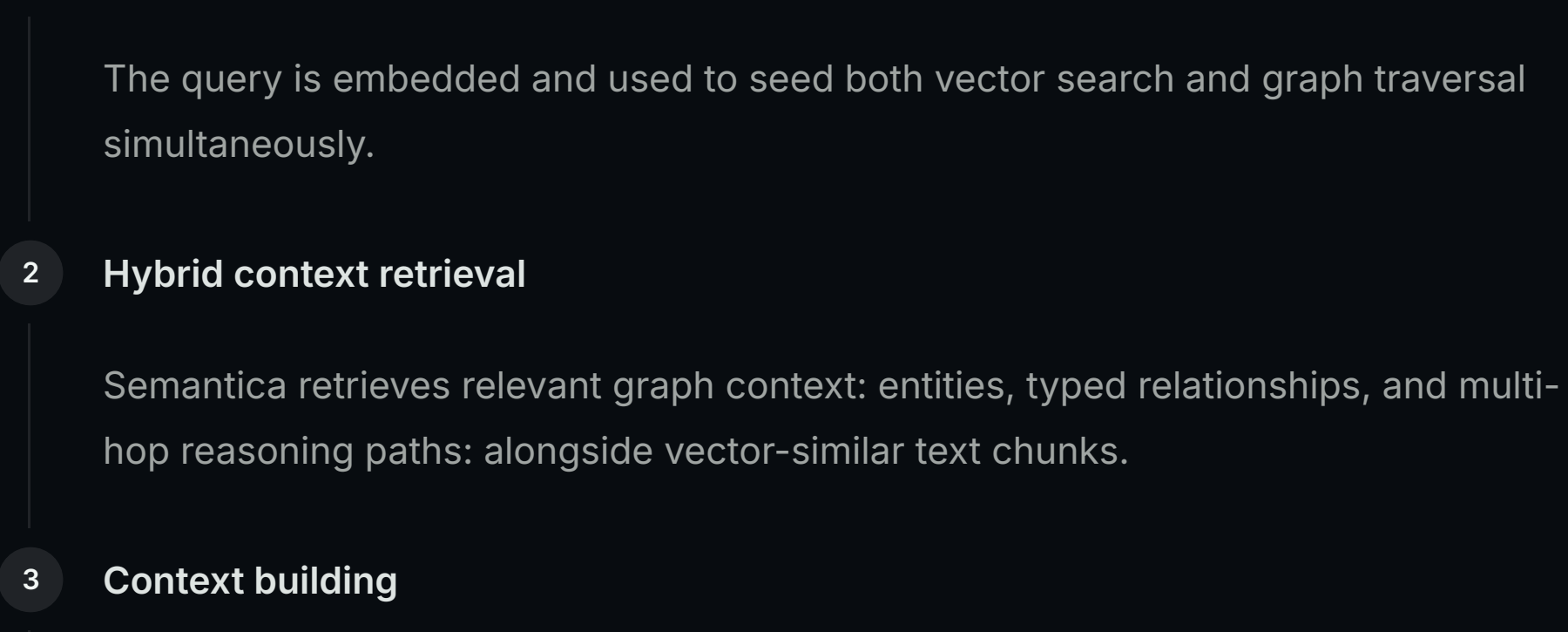
Semantica uses embeddings for:

- **Semantic search:** retrieve by meaning, not just keywords
- **Entity resolution:** match the same entity across different sources
- **Precedent search:** find similar past decisions
- **GraphRAG retrieval:** hybrid vector + graph traversal
- **Distance Intelligence:** N×N semantic distance matrices between any node set

Supported models: Sentence-Transformers, FastEmbed, OpenAI, BGE, Ollama local embeddings.

GraphRAG

GraphRAG (Graph-Augmented Retrieval Augmented Generation) enhances LLM responses by grounding them in a structured knowledge graph rather than raw text chunks alone.



1 **User submits a query**

The query is embedded and used to seed both vector search and graph traversal simultaneously.

2 **Hybrid context retrieval**

Semantica retrieves relevant graph context: entities, typed relationships, and multi-hop reasoning paths; alongside vector-similar text chunks.

3 **Context building**

Retrieved facts and reasoning paths are assembled into a structured prompt context, each fact tagged with its source node and confidence.

4 **LLM generates a grounded response**

The LLM produces an answer where every claim links back to a source node in the graph: no floating assertions, no hallucinations from training data.

🔔 GraphRAG eliminates the **hallucination** and **traceability** problems of standard RAG. Standard RAG retrieves text chunks; GraphRAG retrieves structured facts with typed relationships. The LLM cannot confabulate structure that was never in the graph.

Ontology

An ontology defines the schema and rules for your knowledge: what entity types exist, which relationships are valid, and what constraints apply.

```
ontology = {
  "classes": ["Person", "Organization", "Location"],
  "relationships": ["works_for", "located_in", "founded_by"],
  "rules": {
    "Person":      ["must_have_name"],
    "Organization": ["must_have_name", "can_have_founding_date"]
  }
}
```

Semantica can auto-generate ontologies from your knowledge graph or import existing OWL/RDF/Turtle ontologies. The **Ontology Hub** (v0.5.0) adds a visual editor, SHACL Studio, alignment authoring, and a live health dashboard. See the **Ontology reference** for the full 6-stage generation pipeline.

Reasoning & Inference

Semantica includes multiple reasoning engines to derive new knowledge from existing facts.

```
Known:   Steve Jobs founded Apple Inc.
Known:   Apple Inc. is headquartered in Cupertino
Inferred: Steve Jobs has a connection to Cupertino
```

Forward Chaining

Rete Network

LLM Reasoning

Datalog (v0.4.0)

Engine Comparison

Applies IF/THEN rules repeatedly until no new facts can be derived. Best for alert systems, compliance checks, and trigger-based workflows.

```
from semantica.reasoning import Reasoner

engine = Reasoner()
engine.add_fact("Manager(Alice)")
engine.add_rule("IF Manager(?) THEN HasAuthority(?)")

results = engine.forward_chain() # list of InferenceResult
for r in results:
    print(r.conclusion)           # "HasAuthority(Alice)"
```

`Reasoner.forward_chain()` returns `InferenceResult` objects that carry the rule applied (`rule_used`) and the premises it fired on, and `ExplanationGenerator` turns one into a step-by-step natural-language justification: reasoning here is **not** a black box.

Temporal Intelligence

Knowledge changes over time. Temporal graphs attach `valid_from` / `valid_until` windows to nodes and edges, enabling point-in-time queries and historical analysis.

```
from semantica.kg import TemporalGraphQuery
from datetime import datetime

query_engine = TemporalGraphQuery(enable_temporal_reasoning=True)

# Query the graph as it existed on a specific date
snapshot = query_engine.query_at_time(kg, query="", at_time=datetime(2021, 6, 1))
```

Supported features: Allen interval algebra (all 13 temporal relations), OWL-Time export, `recorded_at` stamping, temporal provenance.

Common uses: tracking company leadership changes, policy evolution, research timelines, financial instrument histories, regulatory compliance windows.

Distance Intelligence

Explore the semantic neighborhood of any entity in your graph: useful for understanding what's conceptually close, detecting clusters, and visualizing knowledge topology.

```
from semantica.kg import SimilarityCalculator

calc = SimilarityCalculator(method="cosine") # "cosine" | "euclidean" | "manhattan"

# Similarity for every unique pair of node embeddings: {(node_a, node_b): score}
pairs = calc.pairwise_similarity({"apple": vec_apple, "google": vec_google, "microsoft": vec_microsoft})

# Or rank a set of embeddings by closeness to one query vector
nearest = calc.find_most_similar(embeddings, query_embedding, top_k=10)
```

Features: N×N semantic distance matrices, ego-mode neighborhood, distance band classification (`direct` / `near` / `mid-range` / `distant`), embedding cache optimization for large graphs.

The **Visualization module** renders distance matrices as interactive heatmaps and ego-mode neighborhood graphs. The **Explorer** embeds distance intelligence directly in the browser dashboard.

Deduplication & Entity Resolution

Real-world data contains the same entity under many names: "Apple", "Apple Inc.", "Apple Computer Inc." Semantica's deduplication pipeline detects these, merges attributes, resolves conflicts, and preserves the original source provenance.

Strategies	Configuration
v1	Jaro-Winkler string similarity Small datasets, fast baseline
blocking_v2	Candidate blocking + similarity Large corpora; reduces O(n²) comparisons
hybrid_v2	Blocking + semantic embedding match Mixed structured/unstructured entity names
semantic_v2	Pure embedding-based resolution Up to 7x faster than v1; handles abbreviations and aliases

Provenance & Auditability

Every fact in Semantica links back to:

- The **source document** it came from
- The **extraction method** used (pattern / ML / LLM)
- The **ontology rules** applied during graph construction
- The **reasoning steps** that produced any inferred fact

🔔 This is W3C PROV-O compliant lineage: suitable for regulated industries that require audit trails (HIPAA, SOX, GDPR, FDA 21 CFR Part 11). `ProvenanceManager.export(format="turtle")` serialises the recorded lineage as PROV-O RDF.

```
from semantica.provenance import ProvenanceManager

prov = ProvenanceManager()
prov.track_entity("apple_inc", source="report.pdf",
  metadata={"extractor": "NamedEntityRecognizer", "confidence": 0.95})

record = prov.get_provenance("apple_inc") # dict; use get_lineage() for the full record
print(record["source_document"])
print(record["timestamp"])
print(record["checksum"])
print(record["metadata"]) # extractor, confidence, and any custom key-value pairs
```

Decision Intelligence

Every agent decision is a first-class object in Semantica: recorded, causally linked, and searchable by precedent. This is the **accountability layer** for AI pipelines: decisions are no longer ephemeral log messages, they are **queryable knowledge** across graph nodes.

```
decision_id = context.record_decision(
  category="model_selection",
  scenario="Choose LLM for production pipeline",
  reasoning="GPT-4 benchmark advantage justifies 3x cost increase",
  outcome="selected_gpt4",
  confidence=0.91,
)

# Find similar past decisions before making a new one
precedents = context.find_precedents("model_selection_reasoning", limit=5)

# Trace downstream impact of a past decision
influence = context.analyze_decision_influence(decision_id)
```

🔔 Use `find_precedents()` before every high-stakes decision. Hybrid similarity search over all recorded decisions surfaces past reasoning that may apply; reducing inconsistency across agent runs and enabling genuine organisational learning from AI decision history.

Conflict Detection

When multiple sources disagree on the same fact, Semantica flags and resolves the conflict rather than silently picking one value.

Resolution strategies:

- **Recency:** prefer the most recent source
- **Source credibility:** prefer the most reliable source (configurable credibility scores)
- **Majority vote:** aggregate across all sources with ≥ 2 agreeing
- **Manual review:** flag for human arbitration; continue pipeline without blocking

See the **Conflicts reference** for `ConflictResolver`, `SourceTracker`, and `InvestigationGuideGenerator`.

Custom Plugin Development

Semantica is designed for extension. Any component: ingestor, extractor, graph builder, reasoning engine: can be replaced or augmented with a custom implementation registered at runtime.

▶ **PluginRegistry:** replace any component by name

▶ **MethodRegistry:** swap a built-in graph operation for your own

- **Quickstart Tutorial:** build a full pipeline with code.
- **Modules Guide:** every module explained with examples.
- **API Reference:** complete technical reference.

< Welcome to Semantica

Modules >



Powered by mintlify